VOICE KEYWORD SPOTTING SYSTEM

[Introduction to Electronics + Elements of Compting1]

AIM – B

TEAM – 04

TEAM MEMBERS :

Devadharshini M	CB.AI.U4AIM24126
Mahashree C	CB.AI.U4AIM24128

Abstract

This project develops an eight-class voice keyword spotting system designed with edge deployment in mind. The system accepts short speech recordings and recognizes one of the commands down, left, no, off, on, right, up, or yes. Raw 16 kHz mono audio is converted into a fixed-length representation and transformed into a 64-bin log-Mel spectrogram. A compact convolutional neural network (CNN) is trained using TensorFlow and Keras. The trained FP32 model is converted to LiteRT and then optimized using post-training full-integer INT8 quantization with a representative dataset of 300 training samples. On the clean test set, the Keras FP32 model achieved 93.62% accuracy, while the INT8 LiteRT model achieved 93.68%. The LiteRT FP32 model occupied 20.71 MB, whereas the INT8 model occupied 5.19 MB, corresponding to a 74.96% reduction. Mean model-invocation latency decreased from 1.5249 ms to 0.4138 ms, giving a measured 3.69× speedup. On a small 40-sample ambient-noise evaluation set, both FP32 and INT8 models achieved 60.00% accuracy, while INT8 remained substantially faster. These results demonstrate that full INT8 quantization can significantly reduce deployment size and inference latency while preserving the classification accuracy of this prototype.

Introduction

Voice keyword spotting is the task of detecting and classifying short spoken commands from audio. It is useful in embedded and edge systems because a device can react to a small vocabulary without continuously sending audio to a remote server. The engineering challenge is to obtain adequate recognition accuracy while keeping the model compact and fast enough for local inference.

The present work focuses on a laboratory-scale prototype. The objective is not to claim clinical, commercial, or production-grade speech recognition performance, but to demonstrate the complete machine-learning and edge-deployment workflow: audio preprocessing, CNN training, FP32 baseline evaluation, LiteRT conversion, INT8 quantization, latency benchmarking, and testing with manually recorded ambient-noise samples.

Problem Statement

Develop a voice keyword spotting system capable of classifying eight predefined spoken commands. The system must convert raw audio into a suitable machine-learning representation, train a classification model, optimize and quantize the model using an edge-deployment toolkit, and compare the resulting models in terms of accuracy, latency, model size, and robustness under real ambient-noise conditions.

Objectives

- Prepare an eight-class speech dataset and create deterministic training, validation, and test splits.
- Convert raw speech waveforms into a fixed-size log-Mel spectrogram representation.
- Train a compact CNN classifier using TensorFlow/Keras.
- Evaluate the FP32 model on a held-out clean test set.
- Convert the trained network to LiteRT format for edge-oriented inference.
- Apply full post-training INT8 quantization using a representative training subset.
- Measure model size and inference latency before and after quantization.

- Evaluate both FP32 and INT8 models using manually recorded ambient-noise samples.
- Document the complete deployment pipeline and discuss limitations and future improvements.

Target Keywords

Class ID	Keyword
0	down
1	left
2	no
3	off
4	on
5	right
6	up
7	yes

The class mapping is fixed throughout preprocessing, training, evaluation, and deployment. The final output layer therefore contains eight logits.

Dataset

The implementation uses the full Speech Commands v0.02 archive and extracts only the eight required command folders. This choice is important because the commonly used TensorFlow mini Speech Commands example contains down, go, left, no, right, stop, up, and yes, rather than the exact eight-command set used in this project. The project therefore does not relabel go or stop as on or off; instead, the exact requested classes are extracted from the full dataset.

Dataset property	Value
Total extracted WAV files	30,794
Classes	8
Sample rate	16,000 Hz
Channels	1 (mono)
Minimum samples	4,778
Maximum samples	16,000
Duration range	0.299–1.000 s
Corrupted files found	0
Train split	24,632 (80%)
Validation split	3,076 (10%)
Test split	3,086 (10%)
Split seed	42

Audio Preprocessing and Feature Extraction

Each waveform is represented at 16 kHz with one channel. The signal is padded or truncated to 16,000 samples so that all examples have a one-second input window. The waveform is then transformed into a time-frequency representation.

- Read a 16 kHz mono WAV recording.
- Pad or truncate the waveform to 16,000 samples.

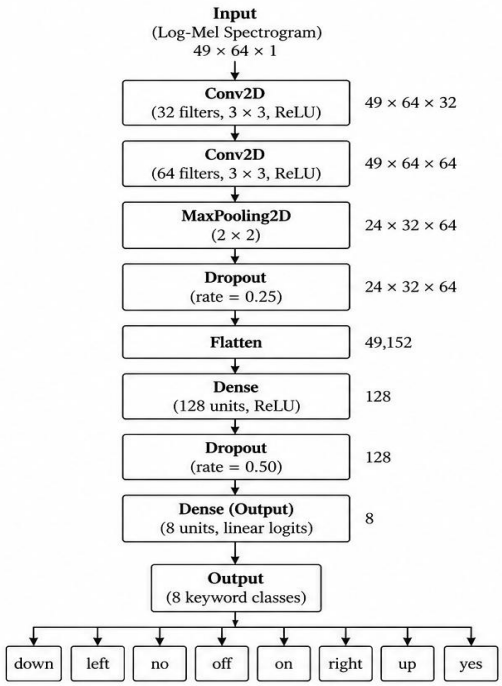
- Compute the short-time Fourier transform (STFT).
- Convert the spectral power into a 64-bin Mel representation.
- Apply logarithmic scaling to obtain a log-Mel spectrogram.
- Normalize each utterance using per-utterance z-score normalization.
- Add a channel dimension to obtain the CNN input shape (49, 64, 1).

Parameter	Setting
Sampling rate	16 kHz
Fixed waveform length	16,000 samples
STFT frame length	640 samples
STFT frame step	320 samples
FFT length	1024
Mel bins	64
Lower frequency	80 Hz
Upper frequency	7,600 Hz
Epsilon	1e-6
Final feature shape	49 × 64 × 1

The final representation preserves short-time frequency information while reducing the dimensionality and variability of the raw waveform. Per-utterance normalization was selected because it requires no global dataset statistics and can be reproduced during deployment.

Model Architecture

A compact CNN was selected to provide a balance between recognition capability and deployment efficiency. Two convolutional layers learn local time-frequency patterns from the log-Mel spectrogram. Max pooling reduces the spatial representation, while dropout provides regularization. A dense layer performs higher-level feature combination before the final eight-class output layer.



Layer (Type)	Configuration	Output Shape
Input	Log-Mel spectrogram (49 × 64 × 1)	49 × 64 × 1
Conv2D (1)	32 filters, 3 × 3, ReLU	49 × 64 × 32
Conv2D (2)	64 filters, 3 × 3, ReLU	49 × 64 × 64
MaxPooling2D	Pool size = 2 × 2	24 × 32 × 64
Dropout (1)	Rate = 0.25	24 × 32 × 64
Flatten	–	49,152
Dense (1)	128 units, ReLU	128
Dropout (2)	Rate = 0.50	128
Dense (Output)	8 units, linear logits	8

Total trainable parameters: 16,280,090

Notes:

1. Input is a 49 × 64 × 1 log-Mel spectrogram computed from 1 second (16,000 samples) of audio.
2. Two convolutional layers are used for feature extraction.
3. Max pooling reduces the spatial dimensions by a factor of 2.
4. Dropout layers are used for regularization.
5. The output layer produces 8 logits corresponding to the keywords: down, left, no, off, on, right, up, yes.
6. The model is trained using Sparse Categorical Cross-Entropy with from_logits=True.

Figure 1: CNN model architecture for 8-class voice keyword spotting.

Layer	Configuration	Output shape / units
Input	Log-Mel spectrogram	49×64×1
Conv2D	32 filters, 3×3, ReLU	49×64×32
Conv2D	64 filters, 3×3, ReLU	49×64×64
MaxPooling2D	2×2	24×32×64
Dropout	rate = 0.25	24×32×64
Flatten	—	49,152
Dense	128 units, ReLU	128
Dropout	rate = 0.50	128
Dense output	8 units, linear logits	8

Total trainable parameters: 16,280,090.

Training Setup

Training parameter	Configuration
Python	3.11.9
TensorFlow	2.21.0
Keras	3.15.1
Optimizer	Adam
Initial learning rate	0.001
Batch size	64
Maximum epochs	20
Loss	Sparse categorical cross-entropy (from_logits=True)
Metric	Accuracy
Random seed	42
Early stopping	Patience = 4, restore best weights
Learning-rate reduction	Factor = 0.5, patience = 2
Checkpointing	Best validation accuracy

The best validation model was restored after early stopping. Training showed a gradual increase in training accuracy followed by a validation plateau, indicating that additional epochs were no longer providing consistent generalization improvement.

FP32 Baseline Evaluation

The trained Keras model was evaluated on all 3,086 samples in the held-out test split. The test set was not used for model fitting or INT8 representative-data calibration.

Metric	Measured result
Test accuracy	93.62%
Test loss	0.2194
Macro F1-score	≈93.58%
Weighted F1-score	≈93.61%
Keras FP32 model file	62.14 MB

The 62.14 MB Keras file is a training artifact and can contain optimizer/training information. For deployment-size comparison, the appropriate comparison is between the LiteRT FP32 and LiteRT INT8 models.

Keyword	Precision	Recall	F1-score	Support
down	95.41%	89.82%	92.53%	393
left	95.07%	91.08%	93.03%	381
no	93.47%	94.18%	93.82%	395
off	91.89%	90.67%	91.28%	375
on	94.97%	93.25%	94.10%	385
right	95.25%	95.25%	95.25%	379
up	88.40%	95.98%	92.03%	373
yes	94.77%	98.52%	96.61%	405

FP32 LiteRT Conversion

The trained Keras model was converted to LiteRT/TFLite format. The converted model was checked using the LiteRT interpreter to verify its input and output tensors and to perform a successful inference.

Property	LiteRT FP32 result
Model size	20.71 MB
Input shape	[1, 49, 64, 1]
Input type	float32
Output shape	[1, 8]
Output type	float32
Inference status	Successful

Quantization Approach

Post-training full integer INT8 quantization was selected because edge deployment benefits from lower memory consumption and reduced computation. The quantization process used TensorFlow LiteRT conversion with default optimization, a representative calibration dataset, built-in INT8 operators, and INT8 input/output tensors.

- Load the trained FP32 Keras model.
- Select 300 samples from the training split as the representative dataset.
- Enable default post-training optimization.
- Restrict the converted model to built-in INT8 operations.
- Set both inference input and inference output types to int8.
- Save and validate the resulting .tflite model.

Quantization setting	Value
Optimization	tf.lite.Optimize.DEFAULT
Representative samples	300 training samples
Representative split	Training only
Supported operators	TFLITE_BUILTINS_INT8
Input inference type	int8
Output inference type	int8
Input scale	0.04296510294
Input zero point	-7
Output scale	0.2194183618
Output zero point	3

The resulting model was confirmed as a fully integer model with INT8 input and output. Quantization therefore affects the deployed model directly rather than merely applying partial weight compression.

End-to-End Deployment Pipeline

The deployment workflow separates model development from on-device inference. During development, the trained Keras model is exported to LiteRT and calibrated using representative training examples. During inference, the same preprocessing steps are applied to incoming audio before the quantized CNN is invoked.

Figure 2. End-to-End Deployment Pipeline

Training, conversion, quantization, and on-device inference workflow

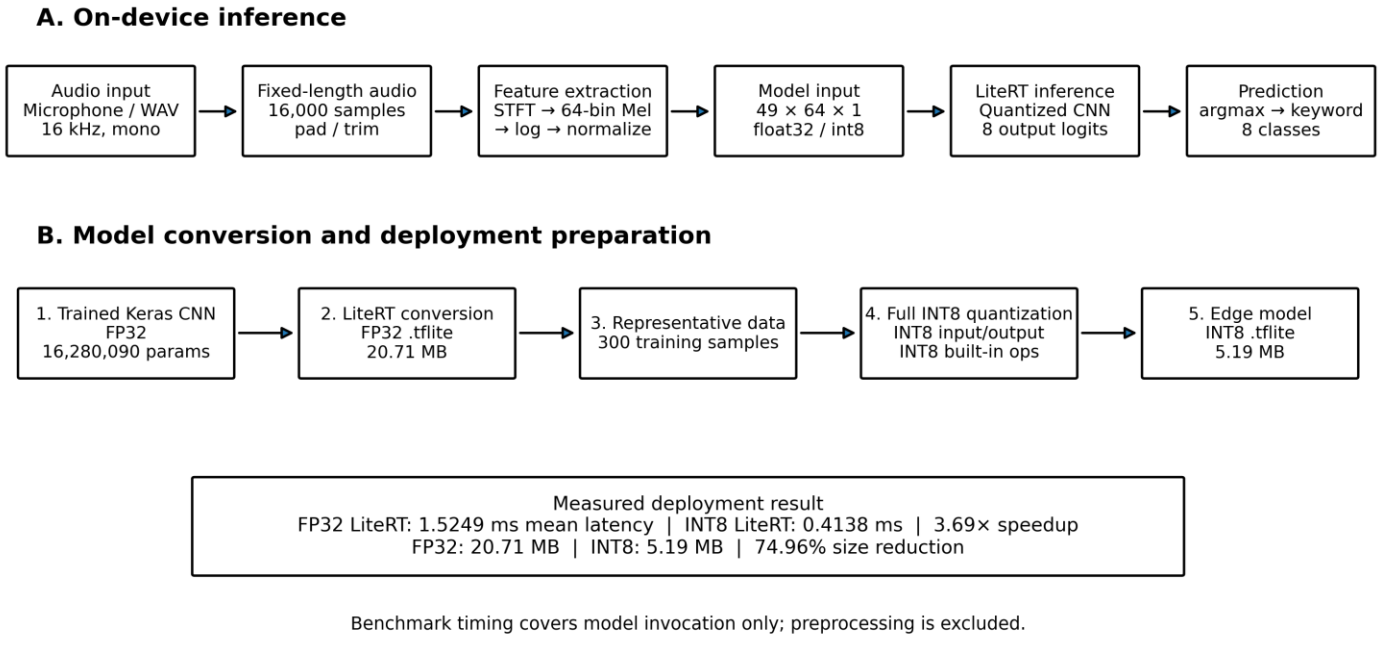


Figure 2. End-to-end training, conversion, quantization, and edge-inference pipeline.

The intended runtime sequence is: audio input → fixed-length waveform → STFT → 64-bin Mel spectrogram → log scaling and normalization → 49×64×1 feature tensor → INT8 LiteRT CNN → eight output logits → predicted keyword.

FP32 vs INT8 Deployment Performance

Metric	LiteRT FP32	LiteRT INT8	Change
Model size	20.71 MB	5.19 MB	74.96% reduction
Mean latency	1.5249 ms	0.4138 ms	72.87% reduction
Median latency	1.4861 ms	0.3795 ms	Lower
p95 latency	1.8483 ms	0.6201 ms	Lower
Throughput	655.78 inf/s	2,416.79 inf/s	≈3.69×
Clean accuracy	93.62%*	93.68%	+0.06 pp

The 93.62% value is the clean-test accuracy of the Keras FP32 baseline. The LiteRT FP32 conversion was verified and benchmarked for deployment latency; the INT8 clean accuracy was evaluated directly through the quantized LiteRT model. The tiny accuracy increase after quantization should be interpreted as normal evaluation variation rather than a claim that quantization improves the classifier.

Latency measurements were obtained with batch size 1, after 20 warm-up invocations and over 200 measured invocations. Timing covers model invocation only; preprocessing and audio capture are excluded. The benchmark therefore isolates the deployment model's inference cost.

Real Ambient-Noise Evaluation

To satisfy the requirement for testing under real ambient conditions, 40 speech recordings were collected using a headset microphone connected to the laptop. Five recordings were collected for each of the eight keywords. The recordings were processed through the same preprocessing pipeline and evaluated using both the FP32 LiteRT and INT8 LiteRT models.

Metric	FP32 LiteRT	INT8 LiteRT
Samples	40	40
Correct predictions	24/40	24/40
Accuracy	60.00%	60.00%
Mean latency	1.6255 ms	0.4993 ms
Median latency	1.5680 ms	0.4774 ms
p95 latency	1.8974 ms	0.6250 ms
Latency reduction	—	69.28%
Speedup	—	3.26×

An important observation is that FP32 and INT8 produced identical predictions for all 40 ambient recordings. Thus, on this small evaluation set, quantization did not change the classification decisions while substantially reducing model-invocation latency.

The 60.00% result should not be generalized to all real-world acoustic environments because the evaluation contains only 40 manually recorded samples and is not a statistically large noise-robustness benchmark. It

nevertheless exposes a meaningful gap between clean test performance and performance under uncontrolled recording conditions.

Error Analysis and Discussion

On the clean test set, the strongest class recall was obtained for yes (98.52%), while right also performed strongly (95.25% recall). The lowest recall was observed for down (89.82%) and off (90.67%). The confusion matrix showed several recurring error patterns, particularly off being confused with up and down being confused with no. These errors suggest that the short acoustic patterns of some commands remain difficult for the compact model to separate.

The INT8 model retained essentially the same clean accuracy as the FP32 baseline. The measured 93.68% INT8 accuracy is only 0.06 percentage points higher than the 93.62% FP32 baseline. This indicates negligible accuracy degradation from the selected quantization procedure.

The ambient-noise results reveal a separate issue: quantization is not the main limitation of the system's robustness. Both models dropped to 60.00% on the small real-noise set, and their predictions were identical. This suggests that improving acoustic robustness would likely require better data diversity, noise augmentation, microphone variation, or a more robust model/preprocessing strategy rather than simply changing numerical precision.

Limitations

- The ambient-noise evaluation contains only 40 recordings and therefore cannot characterize all real-world acoustic conditions.
- The CNN is intentionally compact but contains a relatively large dense layer after flattening, so further architectural optimization is possible.
- The benchmark measures model invocation time and excludes microphone capture, audio buffering, preprocessing, and application-level overhead.
- The experiments were performed on a laptop/CPU environment rather than a specific commercial microcontroller or single-board computer target.
- The project demonstrates engineering feasibility and edge-oriented optimization; it is not a production-grade speech recognition system.

Future Work

- Increase the ambient-noise evaluation set and test multiple environments, distances, microphones, and speaking volumes.
- Add controlled noise augmentation during training using several signal-to-noise ratios.
- Compare the current CNN with depthwise-separable CNNs or other lightweight architectures to reduce parameter count.
- Measure end-to-end latency including preprocessing and audio capture on a real edge device.
- Evaluate power consumption and memory/RAM usage on a target embedded platform.
- Investigate streaming keyword spotting with overlapping audio windows instead of file-based inference.
- Use confusion-matrix-guided data collection to strengthen classes that are frequently confused.

- Evaluate additional quantization/calibration strategies if a specific hardware accelerator requires them.

Conclusion

The project successfully demonstrates a complete voice keyword spotting workflow from raw speech to an edge-oriented quantized model. A compact CNN operating on 49×64 log-Mel spectrograms achieved 93.62% clean-test accuracy in the FP32 Keras baseline. Conversion to LiteRT followed by full INT8 post-training quantization produced a 5.19 MB model compared with a 20.71 MB LiteRT FP32 model, reducing deployment size by 74.96%. Mean model-invocation latency decreased from 1.5249 ms to 0.4138 ms, corresponding to a measured 3.69× speedup.

Most importantly, the quantized model maintained essentially unchanged recognition accuracy, achieving 93.68% on the clean test set and matching the FP32 model on all 40 ambient-noise recordings. The ambient evaluation also showed that environmental robustness is a more significant limitation than quantization for this prototype. Overall, the results support the use of full INT8 LiteRT quantization as an effective edge-deployment optimization for this voice keyword spotting system.

Appendix A. Key Experimental Results

Experiment	Accuracy	Mean latency	Model size
FP32 Keras clean test	93.62%	5.8287 ms	62.14 MB*
FP32 LiteRT clean/deployment benchmark	—	1.5249 ms	20.71 MB
INT8 LiteRT clean test	93.68%	0.4138 ms	5.19 MB
FP32 LiteRT ambient	60.00%	1.6255 ms	20.71 MB
INT8 LiteRT ambient	60.00%	0.4993 ms	5.19 MB